

Habitica API Endpoints and Authentication Guide

Habitica API Reference Guide: Endpoints, Authentication, and Usage

Introduction

Habitica is a gamified productivity platform that transforms daily tasks and goals into an engaging role-playing game. Its robust API enables developers and power users to automate workflows, build integrations, and create third-party tools that interact with user data, tasks, groups, and more. This guide provides a comprehensive, up-to-date reference for the Habitica API, focusing on authentication setup, obtaining API keys, endpoint categories, and practical usage examples. It is designed to serve as both a technical manual and a practical resource for anyone seeking to leverage Habitica’s API for automation, integration, or custom development.

Authentication Setup

Overview of Habitica API Authentication

To ensure secure and authorized access, Habitica’s API requires each request to be authenticated using a combination of user-specific credentials and tool identification headers. The authentication process is straightforward but must be followed precisely to avoid errors or security risks.

Key authentication components:

- **User ID (x-api-user):** A unique identifier for the Habitica user.
- **API Token (x-api-key):** A secret token associated with the user account, functioning similarly to a password.
- **X-Client Header (x-client):** Identifies the client application or script making the request, formatted as UserID-appname.

These credentials are passed as HTTP headers in every API request that requires authentication. The API does not support authentication via query parameters or request bodies; headers are mandatory for all authenticated endpoints.

Required Headers and Formats

The following headers must be included in every authenticated API request:

Header	Value Format	Description
x-api-user	<User ID>	Your unique Habitica User ID

x-api-key	<API Token>	Your secret API Token
x-client	<User ID>-<ToolName>	Identifies your tool; use your User ID and tool/app name
Content-Type	application/json (for JSON requests)	Specifies the request body format (required for POST/PUT/DELETE)

Example:

```
x-api-user: 12345678-90ab-416b-cdef-1234567890ab
x-api-key: 12345678-90ab-416b-cdef-1234567890ab
x-client: 12345678-90ab-416b-cdef-1234567890ab-MyApp
Content-Type: application/json
```

The x-client header is mandatory for all third-party tools and scripts. It helps Habitica staff identify and troubleshoot problematic clients and is required for compliance with API usage guidelines.

Authentication Example

cURL Example:

```
curl https://habitica.com/api/v3/user \
-H "Content-Type: application/json" \
-H "x-api-user: 12345678-90ab-416b-cdef-1234567890ab" \
-H "x-api-key: 12345678-90ab-416b-cdef-1234567890ab" \
-H "x-client: 12345678-90ab-416b-cdef-1234567890ab-Testing"
```

Python Example (requests):

```
import requests

headers = {
    "x-api-user": "12345678-90ab-416b-cdef-1234567890ab",
    "x-api-key": "12345678-90ab-416b-cdef-1234567890ab",
    "x-client": "12345678-90ab-416b-cdef-1234567890ab-MyApp",
    "Content-Type": "application/json"
}

response = requests.get("https://habitica.com/api/v3/user", headers=headers)
print(response.json())
```

JavaScript Example (fetch):

```
fetch('https://habitica.com/api/v3/user', {
  method: 'GET',
  headers: {
    'x-api-user': '12345678-90ab-416b-cdef-1234567890ab',
    'x-api-key': '12345678-90ab-416b-cdef-1234567890ab',
    'x-client': '12345678-90ab-416b-cdef-1234567890ab-MyApp',
```

```
'Content-Type': 'application/json'
}
})
.then(response => response.json())
.then(data => console.log(data));
```

These examples demonstrate the universal requirement for including all three headers in every authenticated request. Omitting any of these headers will result in authentication errors or outright rejection by the API^[1].

Obtaining API Keys

Locating Your User ID and API Token

To interact with the Habitica API, you must first obtain your User ID and API Token. These credentials are unique to your account and should be kept confidential.

On the Habitica Website

1. **Log in** to your Habitica account at habitica.com.
2. Click on your **User Icon** (top right corner).
3. Select **Settings** from the dropdown menu.
4. Navigate to the **API** tab.
5. Here, you will find your **User ID** and **API Token** displayed. You may need to click "Show API Token" to reveal the token.

On the Mobile Apps (iOS/Android)

- Open the app and tap the **Menu** icon.
- Go to **Settings > API**.
- Tap on the **API Token** to copy it to your clipboard.

Security Note:

Your API Token is as sensitive as your password. Never share it publicly or with untrusted parties. If you suspect your token has been compromised, regenerate it via the API settings page or contact Habitica support.

QR Code (Android Only)

- The Android app can generate a QR code containing your User ID (not your API Token). This is used for party invitations and does not grant API access.

Regenerating or Resetting Your API Token

If your API Token is compromised or you wish to reset it:

- On the API settings page, look for an option to **regenerate** your API Token.

- Alternatively, contact Habitica support via the "Report a Bug" feature or by emailing admin@habitica.com.

Resetting your token will immediately invalidate the old token, requiring you to update any scripts or integrations that use it^[2].

API Usage Guidelines

Rate Limiting and Call Frequency

Habitica enforces rate limits to maintain server stability and ensure fair usage among all users and third-party tools. Exceeding these limits will result in HTTP 429 (Too Many Requests) errors.

Key guidelines:

- **Automated scripts** should include a delay of at least 30 seconds between API calls.
- **All API calls** must include the x-client header.
- **Rate limit headers** are included in every API response:
 - X-RateLimit-Limit: Maximum requests allowed in the current period.
 - X-RateLimit-Remaining: Requests left in the current period.
 - X-RateLimit-Reset: Time (in seconds or as a timestamp) until the rate limit resets.

Handling 429 Errors:

- When you receive a 429 error, check the Retry-After header to determine how long to wait before retrying.
- Implement exponential backoff and request queuing to avoid repeated rate limit violations.
- Batch requests and cache responses where possible to minimize API calls^[3].

Example Rate Limit Headers:

X-RateLimit-Limit: 100

X-RateLimit-Remaining: 0

X-RateLimit-Reset: 1699564800

Retry-After: 60

Best Practices:

- Monitor your usage and proactively adjust call frequency.
- Never hardcode API keys in public repositories; use environment variables or secure storage.
- If your tool is used by others, ensure the code is reviewable and follows Habitica's guidelines^[4].

Endpoint Categories Overview

Habitica's API is organized into logical endpoint categories, each serving a distinct aspect of the platform. The main categories are:

- **Tasks:** Manage user and group tasks (habits, dailies, todos, rewards).
- **User:** Access and modify user profile, stats, inventory, and account settings.
- **Groups:** Interact with parties, guilds, and group tasks.
- **Content:** Retrieve static game content (items, pets, mounts, market data).
- **Social:** Manage friends, messages, notifications, and chat.
- **Challenges:** Create, join, and manage challenges.
- **Webhooks:** Set up event-driven integrations.
- **Other:** Miscellaneous endpoints (status, health checks, etc.).

Each category contains multiple endpoints, typically supporting standard HTTP methods (GET, POST, PUT, DELETE) for CRUD operations.

Tasks Endpoints

Tasks are the core of Habitica's productivity system. The API allows full management of all task types, including advanced features like checklists, tags, and scoring.

Task Types

- **habit:** Repetitive behaviors, can be scored up or down.
- **daily:** Recurring tasks, reset daily.
- **todo:** One-time tasks.
- **reward:** Custom rewards users can purchase.

Task Endpoints Table

Method	Path	Description
GET	/tasks/user	Get all tasks for the user
POST	/tasks/user	Create a new user task
GET	/tasks/:taskId	Get a specific task by ID
PUT	/tasks/:taskId	Update a task
DELETE	/tasks/:taskId	Delete a task
POST	/tasks/:taskId/score/:direction	Score (complete/fail) a task
POST	/tasks/:taskId/checklist	Add a checklist item to a task
PUT	/tasks/:taskId/checklist/:itemId	Update a checklist item
DELETE	/tasks/:taskId/checklist/:itemId	Remove a checklist item

POST	/tasks/:taskId/tags/:tagId	Add a tag to a task
DELETE	/tasks/:taskId/tags/:tagId	Remove a tag from a task
POST	/tasks/group/:groupId	Create a group task
GET	/tasks/group/:groupId	Get all tasks for a group
POST	/tasks/:taskId/assign	Assign a task to group members

Checklist and Tag Management:

Checklists and tags are managed via sub-endpoints under each task. Checklists allow for subtasks, while tags help categorize and filter tasks^[5].

Task Completion and Scoring

To score (complete or fail) a task, use:

POST /tasks/:taskId/score/:direction

- :taskId: The unique ID of the task.
- :direction: Either up (complete/positive) or down (fail/negative).

Example:

```
curl -X POST \
-H "x-api-user: <User ID>" \
-H "x-api-key: <API Token>" \
-H "x-client: <User ID>-MyApp" \
https://habitica.com/api/v3/tasks/e4103b44-764b-49e0-a7d0-f93f4f86a9aa/score/up
```

Scoring a task updates user stats (XP, gold, health, mana), task value, streaks, and may trigger random drops or quest progress if in a party quest^[6].

Advanced Task Features

- **Checklists:** Add, update, or remove checklist items for todos and dailies.
- **Tags:** Assign or remove tags for better task organization.
- **Repeating Tasks:** Configure dailies to repeat on specific days.
- **Group Tasks:** Assign tasks to group members, require approval, or set completion conditions.

User Endpoints

User endpoints provide access to profile information, stats, inventory, and account management features.

User Endpoints Table

Method	Path	Description
--------	------	-------------

GET	/user	Get current user profile and stats
PUT	/user	Update user profile
POST	/user/reset	Reset user progress
GET	/user/inventory	Get user inventory (items, pets, mounts)
POST	/user/hatch/:egg/:hatchingPotion	Hatch a pet
POST	/user/equip/:type/:key	Equip an item
POST	/user/purchase/:type/:key	Purchase an item
POST	/user/purchase/gems	Purchase gems
POST	/user/purchase/health-potion	Purchase a health potion
POST	/user/purchase/mystery-set/:key	Purchase a mystery set
POST	/user/purchase/stable/:type/:key	Purchase stable items (pets, mounts)
POST	/user/allocate	Allocate stat points
POST	/user/allocate-now	Auto-allocate stat points
POST	/user/class/change	Change user class
POST	/user/rebirth	Rebirth user (reset level, keep items)

Inventory and Equipment:

Inventory endpoints allow users to manage pets, mounts, gear, and consumables. Equipping and purchasing items is handled via specific endpoints, often requiring item keys found in the /content endpoints.

Groups Endpoints

Groups are central to Habitica's social and collaborative features. The API supports parties, guilds, and group tasks.

Group Types

- **Party:** Small, private group for accountability and quests (max 30 members).
- **Guild:** Larger community, public or private, for shared interests.
- **Tavern:** Global chat group, all users are members by default.

Groups Endpoints Table

Method	Path	Description
POST	/groups	Create a new group (party or guild)
GET	/groups	Get all groups for the user
GET	/groups/:groupId	Get details of a specific group
PUT	/groups/:groupId	Update group details
POST	/groups/:groupId/join	Join a group
POST	/groups/:groupId/leave	Leave a group

POST	/groups/:groupId/invite	Invite users to a group
POST	/groups/:groupId/ removeMember/:memberId	Remove a member from a group
POST	/groups/:groupId/quests/invite/:questKey	Invite group to a quest
POST	/tasks/group/:groupId	Create a group task
GET	/tasks/group/:groupId	Get all group tasks
POST	/tasks/:taskId/assign	Assign a group task to members

Group Management:

Group leaders and managers have special permissions to edit group details, assign tasks, and manage members. Group plans (subscriptions) unlock advanced features like shared task boards and approval workflows^[5].

Content Endpoints

Content endpoints provide access to static game data, such as items, pets, mounts, and market offerings. These are essential for building tools that need to display or interact with Habitica's in-game content.

Content Endpoints Table

Method	Path	Description
GET	/content	Get all available content objects
GET	/content/:type	Get specific content type (e.g., pets)

Content Types:

- Items (gear, potions, food)
- Pets and mounts
- Market offerings
- Quests and achievements

The /content endpoint returns comprehensive data structures, including keys and metadata needed for inventory and purchase endpoints^[7].

Social Endpoints

Social endpoints enable interaction with other users, including friends, messages, notifications, and chat.

Social Endpoints Table

Method	Path	Description
GET	/members/:memberId	Get another user's public profile
POST	/members/:memberId/message	Send a private message
GET	/user/inbox/messages	Get user's messages
POST	/user/inbox/messages/:messageId/ flag	Flag a message
POST	/user/inbox/messages/:messageId/ like	Like a message
GET	/user/notifications	Get user notifications
POST	/user/notifications/read	Mark notifications as read
GET	/groups/:groupId/chat	Get group chat messages
POST	/groups/:groupId/chat	Post a message to group chat
POST	/groups/:groupId/chat/:chatId/like	Like a group chat message
POST	/groups/:groupId/chat/:chatId/flag	Flag a group chat message

Notifications and Messaging:

Notifications inform users of important events (e.g., quest invites, task approvals). Messaging endpoints support private and group communication, with moderation features like flagging and liking messages^[5].

Challenges Endpoints

Challenges are competitive or collaborative events within groups or the wider Habitica community. The API supports full challenge lifecycle management.

Challenges Endpoints Table

Method	Path	Description
POST	/challenges	Create a new challenge
GET	/challenges	Get all challenges
GET	/challenges/:challengeId	Get challenge details
POST	/challenges/:challengeId/join	Join a challenge
POST	/challenges/:challengeId/leave	Leave a challenge
POST	/challenges/:challengeId/close	Close a challenge
POST	/challenges/:challengeId/selectWinner	Select a winner for a challenge

Challenge Data:

Challenge endpoints allow for participant management, task tracking, and winner selection.

Tools like the Challenge Data Tool use these endpoints to analyze participation and award prizes^[8].

Webhooks and Third-Party Integrations

Habitica supports webhooks for event-driven integrations. Webhooks notify external scripts or services when specific events occur (e.g., task completion, chat messages).

Webhook Setup

1. **Create an external script** to receive POST requests (e.g., using Flask, Node.js, or Google Apps Script).
2. **Register the webhook URL** in Habitica:
 - Go to **Settings > API** on the Habitica website.
 - Add your webhook URL under the Webhooks section.

Technical Details:

- Habitica sends a POST request with event data to your webhook URL.
- The webhook must respond with a 2xx HTTP code within 30 seconds.
- After 10 consecutive failures in a month, the webhook is disabled.

Example (Python Flask):

```
from flask import Flask, request
app = Flask(__name__)

@app.route('/taskevent', methods=['POST'])
def webhook():
    print(request.json) # Handle event data here
    return "", 200
```

```
if __name__ == "__main__":
    app.run(host='0.0.0.0', port=5000)
```

Security Note:

Ensure your webhook endpoint is secured and does not expose sensitive data. Only process requests from Habitica's servers^[10].

Error Handling and Response Structure

API Responses

- **Success:**
 - HTTP status code: 2xx

- JSON body: { "success": true, "data": ... }
- **Error:**
 - HTTP status code: 4xx or 5xx
 - JSON body: { "success": false, "error": "ErrorType", "message": "Description" }

Common Error Codes:

- 401 Unauthorized: Invalid or missing credentials.
- 403 Forbidden: Insufficient permissions.
- 404 Not Found: Resource does not exist.
- 429 Too Many Requests: Rate limit exceeded.
- 500 Internal Server Error: Server-side issue.

Error Handling Best Practices:

- Always check the success property in the response.
- For 429 errors, respect the Retry-After header before retrying.
- Log and monitor error messages for troubleshooting.
- Never expose API tokens or sensitive data in error logs or public outputs^[3].

Security Best Practices

- **Protect your API Token:** Treat it as a password; never share or commit it to public repositories.
- **Use environment variables** or secure credential storage for scripts and applications.
- **Rotate API tokens** if you suspect compromise.
- **Monitor rate limits** and avoid excessive or automated requests that could trigger bans.
- **Review third-party tools** before granting them access to your account.
- **Implement proper error handling** to avoid leaking sensitive information.

Versioning and Deprecated Endpoints

API Versioning

- **Current Stable Version:** v3 (/api/v3/)
- **Version 4:** In development, not recommended for use. Its behavior may change without notice, and access may be blocked at any time.
- **Older Versions:** v1 and v2 are deprecated and disabled.

Guidance:

Always use the /api/v3/ endpoints for all integrations and tools. Ignore any references to v4 in the documentation until it is officially released and stable.

Example Requests: Common Use Cases

1. Get Current User Profile

Endpoint: GET /user

cURL:

```
curl https://habitica.com/api/v3/user \
-H "x-api-user: <User ID>" \
-H "x-api-key: <API Token>" \
-H "x-client: <User ID>-MyApp"
```

2. Create a New To-Do Task

Endpoint: POST /tasks/user

Request Body:

```
{
  "type": "todo",
  "text": "Write research report",
  "notes": "Complete the Habitica API guide",
  "tags": ["tag-id-1", "tag-id-2"],
  "checklist": [
    { "text": "Draft outline", "completed": false },
    { "text": "Write content", "completed": false }
  ]
}
```

cURL:

```
curl -X POST https://habitica.com/api/v3/tasks/user \
-H "Content-Type: application/json" \
-H "x-api-user: <User ID>" \
-H "x-api-key: <API Token>" \
-H "x-client: <User ID>-MyApp" \
-d '{
  "type": "todo",
  "text": "Write research report",
  "notes": "Complete the Habitica API guide",
  "tags": ["tag-id-1", "tag-id-2"],
  "checklist": [
    { "text": "Draft outline", "completed": false },
```

```
{ "text": "Write content", "completed": false }  
]  
'
```

3. Score (Complete) a Task

Endpoint: POST /tasks/:taskId/score/up

cURL:

```
curl -X POST https://habitica.com/api/v3/tasks/<taskId>/score/up \  
-H "x-api-user: <User ID>" \  
-H "x-api-key: <API Token>" \  
-H "x-client: <User ID>-MyApp"
```

4. Get All Group Tasks

Endpoint: GET /tasks/group/:groupId

cURL:

```
curl https://habitica.com/api/v3/tasks/group/<groupId> \  
-H "x-api-user: <User ID>" \  
-H "x-api-key: <API Token>" \  
-H "x-client: <User ID>-MyApp"
```

5. Join a Challenge

Endpoint: POST /challenges/:challengeId/join

cURL:

```
curl -X POST https://habitica.com/api/v3/challenges/<challengeId>/join \  
-H "x-api-user: <User ID>" \  
-H "x-api-key: <API Token>" \  
-H "x-client: <User ID>-MyApp"
```

Endpoint Reference Tables

Tasks Endpoints

Method	Path	Description
GET	/tasks/user	Get all user tasks
POST	/tasks/user	Create a new user task
GET	/tasks/:taskId	Get a specific task
PUT	/tasks/:taskId	Update a task
DELETE	/tasks/:taskId	Delete a task
POST	/tasks/:taskId/score/:direction	Score a task (up/down)

POST	/tasks/:taskId/checklist	Add checklist item
PUT	/tasks/:taskId/checklist/:itemId	Update checklist item
DELETE	/tasks/:taskId/checklist/:itemId	Remove checklist item
POST	/tasks/:taskId/tags/:tagId	Add tag to task
DELETE	/tasks/:taskId/tags/:tagId	Remove tag from task
POST	/tasks/group/:groupId	Create a group task
GET	/tasks/group/:groupId	Get group tasks
POST	/tasks/:taskId/assign	Assign task to group members

User Endpoints

Method	Path	Description
GET	/user	Get user profile and stats
PUT	/user	Update user profile
POST	/user/reset	Reset user progress
GET	/user/inventory	Get user inventory
POST	/user/hatch/:egg/:hatchingPotion	Hatch a pet
POST	/user/equip/:type/:key	Equip an item
POST	/user/purchase/:type/:key	Purchase an item
POST	/user/purchase/gems	Purchase gems
POST	/user/purchase/health-potion	Purchase health potion
POST	/user/purchase/mystery-set/:key	Purchase mystery set
POST	/user/purchase/stable/:type/:key	Purchase stable items
POST	/user/allocate	Allocate stat points
POST	/user/allocate-now	Auto-allocate stat points
POST	/user/class/change	Change user class
POST	/user/rebirth	Rebirth user

Groups Endpoints

Method	Path	Description
POST	/groups	Create a new group
GET	/groups	Get user's groups
GET	/groups/:groupId	Get group details
PUT	/groups/:groupId	Update group
POST	/groups/:groupId/join	Join group
POST	/groups/:groupId/leave	Leave group
POST	/groups/:groupId/invite	Invite users
POST	/groups/:groupId/removeMember/:memberId	Remove member

POST	/groups/:groupId/quests/invite/:questKey	Invite to quest
POST	/tasks/group/:groupId	Create group task
GET	/tasks/group/:groupId	Get group tasks
POST	/tasks/:taskId/assign	Assign group task

Challenges Endpoints

Method	Path	Description
POST	/challenges	Create challenge
GET	/challenges	Get all challenges
GET	/challenges/:challengeId	Get challenge details
POST	/challenges/:challengeId/join	Join challenge
POST	/challenges/:challengeId/leave	Leave challenge
POST	/challenges/:challengeId/close	Close challenge
POST	/challenges/:challengeId/selectWinner	Select winner

Conclusion

The Habitica API is a powerful tool for automating productivity workflows, building integrations, and enhancing the Habitica experience. By following the authentication procedures, respecting rate limits, and leveraging the comprehensive set of endpoints, developers can create robust tools and scripts that interact seamlessly with Habitica's gamified ecosystem. Always prioritize security, monitor your usage, and stay updated with the latest API documentation and guidelines to ensure reliable and secure integrations.

For further details, consult the official Habitica API documentation and community resources. This guide will remain a living document, updated as Habitica's API evolves and new features are introduced.

References (10)

1. *How to Pass x-API-key in Header? - apidog.com.* <https://apidog.com/blog/pass-x-api-key-header/>
2. *habitica/docs/authentication_troubleshooting.md at branch-2*
https://github.com/samarshirazi/habitica/blob/branch-2/docs/authentication_troubleshooting.md
3. *HTTP Error 429 (Too Many Requests) - How to Fix.* <https://blog.postman.com/http-error-429/>
4. *Habitica Task Adder - GitHub.* https://github.com/martinedb/habitica_task_adder
5. *Group System | HabitRPG/habitica | DeepWiki.* <https://deepwiki.com/HabitRPG/habitica/6-group-system>

6. *Task Scoring & Rewards* | *HabitRPG/habituca* | *DeepWiki*.
<https://deepwiki.com/HabitRPG/habituca/5.2-task-scoring-and-rewards>
7. *GitHub - delightedCrow/HabitucaMagic: A convenient way to interact with*
<https://github.com/delightedCrow/HabitucaMagic>
8. *Challenge Data Tool* | *Habituca Wiki* | *Fandom*.
https://habituca.fandom.com/wiki/Challenge_Data_Tool
9. *Event-Driven (Webhook) Scripts Setup Guide - Habituca Wiki*.
https://habituca.fandom.com/wiki/Event-Driven_%28Webhook%29_Scripts_Setup_Guide